题库管理系统接口文档

0. 通用说明

0.1 Base URL

- 统一前缀：`/api`

0.2 认证与鉴权（JWT + RBAC）

- 登录成功后服务端返回 `token`（JWT）。后续请求在 Header 中携带：`Authorization: Bearer <token>`。
- 角色：`STUDENT` / `TEACHER` / `ADMIN`。

0.3 通用返回结构

除登录接口外，推荐统一返回：

```
{
  "success": true,
  "msg": "ok",
  "data": {}
}
```

当发生业务错误时：

```
{
  "success": false,
  "msg": "错误原因",
  "code": "ERROR_CODE"
}
```

0.4 分页返回结构

列表接口默认返回：

```
{
  "success": true,
  "msg": "ok",
```

```
  "data": {
    "page": 1,
    "size": 10,
    "total": 0,
    "list": []
  }
}
```

0.5 关键枚举值（和数据库/代码保持一致）

- `questionType`：1=single，2=multiple，3=true_false，4=blank，5=short，6=code，7=code_reading
- `difficulty`：1~5
- `questionStatus`：1=draft，2=published，3=archived
- `analysisSource`：1=manual，2=llm_draft，3=llm_final
- `assignmentPublishStatus`：1=draft，2=published，3=closed
- `attemptType`：1=assignment，2=practice
- `attemptStatus`：1=in_progress，2=submitted，3=grading，4=graded
- `answerStatus`：1=draft，2=submitted
- `gradingMode`：1=auto，2=llm，3=manual
- `appealStatus`：1=pending，2=approved，3=rejected，4=resolved

0.6 相比开题报告新增的表/概念说明（重点：`qb_llm_call`）

你的开题报告里已经明确：**主观题用 LLM 结构化 JSON 输出（得分/扣分点/理由/置信度）**，并且要**保留模型原始输出与评分依据**用于复核与申诉。
因此我在数据库与接口层补齐了“证据链 + 可追溯性”相关表：

- `qb_llm_call`：保存一次 LLM 调用的**prompt、原始 response、结构化 response_json、耗时、token、状态**，并通过 `biz_type/biz_id` 关联业务（题目解析生成/主观题评分等）。
- `qb_grading_record.llm_call_id`：把某次评分记录与 LLM 调用绑定，教师复核时能直接看到“模型怎么判的”。
- `qb_attempt_question.snapshot_json / qb_paper_question.snapshot_json`：**题目快照**。发布后题目若被修改，历史作答仍可追溯。
- `qb_assignment_target`：支持“只给某些学生发布作业”（有些课程场景需要）。
- `qb_practice_rule / qb_practice_generation`：自适应抽题需要可配置规则与可追溯生成过程（调参、复现实验很有用）。

- `sys_audit_log / sys_login_log`：用于后台审计（谁在什么时候改了什么、登录是否成功等）。
- `qb_question_case`：为编程/代码题预留测试样例（可选加分点）。

这些扩展不会改变你“题库-组卷-作答-判分-统计-申诉/答疑”的主体结构，只是把工程里必要的可追溯性补齐。

1. 用户与权限

1.1. 用户登录

请求路径	**/api/login**
请求方式	POST
请求参数	application/json

> 需要登录：否

请求参数：

参数名	类型	是否必须	备注
username	string	是	用户名
password	string	是	密码

返回数据：

```
{
  "success": true,
  "token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...",
  "user": {
    "id": 10001,
    "username": "student01",
    "displayName": "张三",
    "roles": [
      "STUDENT"
    ]
  },
  "msg": "登录成功"
}
```

1.2. 用户退出

请求路径	/api/logout
请求方式	POST
请求参数	无

> 需要登录：是（JWT）

> 权限要求：STUDENT / TEACHER / ADMIN

> 说明：前后端若使用 JWT 无状态方案，后端可不保存 session；此接口主要用于前端清理本地 token 或做黑名单（可选）。

请求参数：

无

返回数据：

```
{
  "success": true,
  "msg": "退出成功",
  "data": null
}
```

1.3. 获取当前登录用户信息

请求路径	/api/auth/me
请求方式	GET
请求参数	无

> 需要登录：是（JWT）

> 权限要求：STUDENT / TEACHER / ADMIN

请求参数：

无

返回数据：

```
{
  "success": true,
  "msg": "ok",
  "data": {
    "id": 10001,
    "username": "student01",
    "displayName": "张三",
    "email": "student01@example.com",
    "roles": [
      "STUDENT"
    ]
  }
}
```

1.4. 管理员-用户列表（分页）

请求路径	/api/admin/users
请求方式	GET
请求参数	query

> 需要登录：是（JWT）

> 权限要求：ADMIN

请求参数：

参数名	类型	是否必须	备注
keyword	string	否	用户名/昵称关键字
roleCode	string	否	角色筛选：STUDENT/TEACHER/ADMIN
status	int	否	1=active,0=disabled
page	int	否	页码（默认1）
size	int	否	每页条数（默认10）

返回数据：

```json
{
  "success": true,
  "msg": "ok",
  "data": {
    "page": 1,
    "size": 10,
    "total": 2,
    "list": [
      {
        "id": 10001,
        "username": "student01",
        "displayName": "张三",
        "email": "student01@example.com",
        "status": 1,
        "roles": [
          "STUDENT"
        ]
      },
      {
        "id": 20001,
        "username": "teacher01",
        "displayName": "李老师",
        "email": "teacher01@example.com",
        "status": 1,
        "roles": [
          "TEACHER"
        ]
      }
    ]
  }
}
```

1.5. 管理员-创建用户

请求路径	/api/admin/users
请求方式	POST
请求参数	application/json

> 需要登录：是（JWT）

> 权限要求：ADMIN

请求参数：

参数名	类型	是否必须	备注
username	string	是	用户名（唯一）
password	string	是	初始密码
displayName	string	否	显示名
email	string	否	邮箱
status	int	否	1=active,0=disabled（默认1）
roleCodes	array	是	角色数组，如 ["STUDENT"]

返回数据：

```
{
  "success": true,
  "msg": "创建成功",
  "data": {
    "userId": 30001
  }
}
```

1.6. 管理员-更新用户基础信息

请求路径	/api/admin/users/{userId}
请求方式	PUT
请求参数	application/json

> 需要登录：是（JWT）

> 权限要求：ADMIN

请求参数：

参数名	类型	是否必须	备注
userId	long(path)	是	用户ID
displayName	string	否	显示名
email	string	否	邮箱
status	int	否	1=active,0=disabled

返回数据：

```
{
  "success": true,
  "msg": "更新成功",
  "data": null
}
```

1.7. 管理员-设置用户角色

请求路径	/api/admin/users/{userId}/roles
请求方式	PUT
请求参数	application/json

> 需要登录：是（JWT）

> 权限要求：ADMIN

请求参数：

参数名	类型	是否必须	备注
userId	long(path)	是	用户ID
roleCodes	array	是	角色数组，如 ["TEACHER"]

返回数据：

```
{
  "success": true,
  "msg": "设置成功",
```

```
  "data": null
}
```

2. 标签管理

2.1. 获取标签列表（扁平）

请求路径	/api/tags
请求方式	GET
请求参数	query

> 需要登录：是（JWT）

> 权限要求：STUDENT / TEACHER / ADMIN

请求参数：

参数名	类型	是否必须	备注
keyword	string	否	按名称模糊搜索
tagType	int	否	1=knowledge,2=chapter,3=custom
parentId	long	否	父标签ID（可选）

返回数据：

```
{
  "success": true,
  "msg": "ok",
  "data": [
    {
      "id": 1,
      "tagName": "指针",
      "tagType": 1,
      "parentId": null,
      "tagLevel": 1
    },
    {
      "id": 2,
      "tagName": "数组",
```

```
      "tagType": 1,
      "parentId": null,
      "tagLevel": 1
    }
  ]
}
```

2.2. 获取标签树（用于树形选择器）

请求路径	/api/tags/tree
请求方式	GET
请求参数	query

> 需要登录：是（JWT）

> 权限要求：STUDENT / TEACHER / ADMIN

请求参数：

参数名	类型	是否必须	备注
tagType	int	否	1=knowledge,2=chapter,3=custom；不传则返回全部

返回数据：

```
{
  "success": true,
  "msg": "ok",
  "data": [
    {
      "id": 1,
      "tagName": "指针",
      "children": [
        {
          "id": 11,
          "tagName": "指针算术",
          "children": []
        }
      ]
    }
```

```
  ]
}
```

2.3. 创建标签

请求路径	/api/tags
请求方式	POST
请求参数	application/json

> 需要登录：是（JWT）

> 权限要求：TEACHER / ADMIN

请求参数：

参数名	类型	是否必须	备注
tagName	string	是	标签名称（唯一）
tagCode	string	否	标签编码（可选）
parentId	long	否	父标签ID（可选）
tagType	int	是	1=knowledge,2=chapter,3=custom
sortOrder	int	否	排序值（默认0）

返回数据：

```
{
  "success": true,
  "msg": "创建成功",
  "data": {
    "tagId": 101
  }
}
```

2.4. 更新标签

请求路径	/api/tags/{tagId}
请求方式	PUT
请求参数	application/json

> 需要登录：是（JWT）

> 权限要求：TEACHER / ADMIN

请求参数：

参数名	类型	是否必须	备注
tagId	long(path)	是	标签ID
tagName	string	否	标签名称
tagCode	string	否	标签编码
parentId	long	否	父标签ID
tagType	int	否	1=knowledge,2=chapter,3=custom
sortOrder	int	否	排序值

返回数据：

```
{
  "success": true,
  "msg": "更新成功",
  "data": null
}
```

2.5. 删除标签（软删除）

请求路径	/api/tags/{tagId}
请求方式	DELETE
请求参数	无

> 需要登录：是（JWT）

> 权限要求：TEACHER / ADMIN

> 说明：若该标签已被题目/帖子引用，建议后端做引用校验：要么禁止删除，要么级联解绑。

请求参数：

参数名	类型	是否必须	备注
tagId	long(path)	是	标签ID

返回数据：

```
{
  "success": true,
  "msg": "删除成功",
  "data": null
}
```

3. 题目管理

3.1. 创建题目（支持选项、标签、代码题样例）

请求路径	/api/questions
请求方式	POST
请求参数	application/json

> 需要登录：是（JWT）

> 权限要求：TEACHER / ADMIN

请求参数：

参数名	类型	是否必须	备注
title	string	否	题目标题（可选）
questionType	int	是	题型：1~7
difficulty	int	是	难度：1~5

参数名	类型	是否必须	备注
chapter	string	否	章节（可选）
stem	string	是	题干（支持富文本/Markdown）
standardAnswer	string	否	标准答案（客观题/参考答案）
answerFormat	int	否	1=text,2=json（默认1）
analysisText	string	否	解析（可先空，后续可 LLM 生成初稿）
status	int	否	1=draft,2=published（默认1）
tagIds	array	否	绑定的标签ID数组
options	array	否	选择题/判断题选项（非选择题可不传）
options[].optionLabel	string	否	选项标记：A/B/C...
options[].optionContent	string	否	选项内容
options[].isCorrect	boolean	否	是否正确答案
options[].sortOrder	int	否	选项排序
cases	array	否	编程题样例/测试点（可选）
cases[].caseNo	int	否	用例编号（从1开始）
cases[].inputData	string	否	输入数据
cases[].expectedOutput	string	否	期望输出
cases[].caseScore	int	否	该用例分值（可选）
cases[].isSample	boolean	否	是否样例（展示给学生）

返回数据：

```
{
  "success": true,
  "msg": "创建成功",
  "data": {
    "questionId": 50001
  }
}
```

3.2. 更新题目（含选项/标签/样例）

请求路径	/api/questions/{questionId}
请求方式	PUT
请求参数	application/json

> 需要登录：是（JWT）

> 权限要求：TEACHER / ADMIN

> 说明：建议后端实现“全量覆盖”策略：options/tagIds/cases 以请求内容为准（先删后插），避免前端维护差量过复杂。

请求参数：

参数名	类型	是否必须	备注
questionId	long(path)	是	题目ID
title	string	否	题目标题
questionType	int	否	题型：1~7
difficulty	int	否	难度：1~5
chapter	string	否	章节
stem	string	否	题干
standardAnswer	string	否	标准答案
answerFormat	int	否	1=text,2=json
analysisText	string	否	解析
status	int	否	1=draft,2=published,3=archived
tagIds	array	否	标签ID数组（全量覆盖）
options	array	否	选项（全量覆盖）
cases	array	否	代码题用例（全量覆盖）

返回数据：

```
{
  "success": true,
  "msg": "更新成功",
```

```
  "data": null
}
```

3.3. 删除题目（软删除）

请求路径	/api/questions/{questionId}
请求方式	DELETE
请求参数	无

> 需要登录：是（JWT）

> 权限要求：TEACHER / ADMIN

> 说明：若题目已被试卷引用，建议禁止删除或仅允许归档（status=archived），避免历史追溯问题。

请求参数：

参数名	类型	是否必须	备注
questionId	long(path)	是	题目ID

返回数据：

```
{
  "success": true,
  "msg": "删除成功",
  "data": null
}
```

3.4. 获取题目详情（教师端：含标准答案/解析）

请求路径	/api/questions/{questionId}
请求方式	GET
请求参数	无

> 需要登录：是（JWT）

> 权限要求：TEACHER / ADMIN

请求参数：

参数名	类型	是否必须	备注
questionId	long(path)	是	题目ID

返回数据：

```
{
  "success": true,
  "msg": "ok",
  "data": {
    "id": 50001,
    "title": "指针与数组",
    "questionType": 1,
    "difficulty": 3,
    "chapter": "C语言-指针",
    "stem": "以下代码输出是什么？ ...",
    "standardAnswer": "B",
    "analysisText": "解析： ...",
    "analysisSource": 1,
    "analysisLlmCallId": null,
    "status": 1,
    "tagIds": [
      1,
      11
    ],
    "options": [
      {
        "optionLabel": "A",
        "optionContent": "xxx",
        "isCorrect": false,
        "sortOrder": 1
      },
      {
        "optionLabel": "B",
        "optionContent": "yyy",
        "isCorrect": true,
        "sortOrder": 2
      }
```

```
    ],
    "cases": []
  }
}
```

3.5. 题目检索（分页，多条件）

请求路径	/api/questions
请求方式	GET
请求参数	query

> 需要登录：是（JWT）

> 权限要求：STUDENT / TEACHER / ADMIN

> 说明：学生端默认只返回 published 题目；教师端可查 draft/published/archived。

请求参数：

参数名	类型	是否必须	备注
keyword	string	否	关键字（title/stem 模糊）
tagId	long	否	单个标签筛选（可选）
tagIds	array	否	多个标签筛选（可选，AND/OR 由后端定义）
questionType	int	否	题型：1~7
difficulty	int	否	难度：1~5
chapter	string	否	章节筛选
status	int	否	1=draft,2=published,3=archived（教师端可用）
page	int	否	页码（默认1）
size	int	否	每页条数（默认10）

返回数据：

```
{
  "success": true,
  "msg": "ok",
  "data": {
```

```
    "page": 1,
    "size": 10,
    "total": 1,
    "list": [
      {
        "id": 50001,
        "title": "指针与数组",
        "questionType": 1,
        "difficulty": 3,
        "chapter": "C语言-指针",
        "status": 2,
        "tagIds": [
          1,
          11
        ]
      }
    ]
  }
}
```


3.6. 发布题目（draft -> published）

请求路径	/api/questions/{questionId}/publish
请求方式	POST
请求参数	无

> 需要登录：是（JWT）

> 权限要求：TEACHER / ADMIN

请求参数：

参数名	类型	是否必须	备注
questionId	long(path)	是	题目ID

返回数据：


```
{
  "success": true,
```

```
  "msg": "发布成功",
  "data": null
}
```

3.7. LLM 生成题目解析初稿（千问）

请求路径	/api/questions/{questionId}/analysis/llm
请求方式	POST
请求参数	application/json

> 需要登录：是（JWT）

> 权限要求：TEACHER / ADMIN

> 说明：后端会写入 qb_llm_call（biz_type=QUESTION_ANALYSIS），并把 llm_call_id 记录到 qb_question.analysis_llm_call_id。

请求参数：

参数名	类型	是否必须	备注
questionId	long(path)	是	题目ID
forceRegenerate	boolean	否	是否强制重新生成（默认false）
modelName	string	否	模型名称（默认 qwen-plus）
temperature	float	否	采样温度（默认0.2~0.7，按需）

返回数据：

```
{
  "success": true,
  "msg": "生成成功",
  "data": {
    "questionId": 50001,
    "analysisText": "【LLM 初稿】...",
    "analysisSource": 2,
    "analysisLlmCallId": 90001
  }
}
```

3.8. 获取题目代码用例列表

请求路径	/api/questions/{questionId}/cases
请求方式	GET
请求参数	无

> 需要登录：是（JWT）

> 权限要求：TEACHER / ADMIN

请求参数：

参数名	类型	是否必须	备注
questionId	long(path)	是	题目ID

返回数据：

```
{
  "success": true,
  "msg": "ok",
  "data": [
    {
      "id": 70001,
      "caseNo": 1,
      "inputData": "1 2",
      "expectedOutput": "3",
      "caseScore": 10,
      "isSample": true
    },
    {
      "id": 70002,
      "caseNo": 2,
      "inputData": "10 20",
      "expectedOutput": "30",
      "caseScore": 10,
      "isSample": false
    }
```

```
  ]
}
```

3.9. 新增/更新题目代码用例（单条）

请求路径	/api/questions/{questionId}/cases
请求方式	POST
请求参数	application/json

> 需要登录：是（JWT）

> 权限要求：TEACHER / ADMIN

> 说明：建议后端按 (question_id, case_no) 唯一键 upsert，避免重复编号。

请求参数：

参数名	类型	是否必须	备注
questionId	long(path)	是	题目ID
caseNo	int	是	用例编号（从1开始）
inputData	string	是	输入数据
expectedOutput	string	是	期望输出
caseScore	int	否	用例分值
isSample	boolean	否	是否样例（默认false）

返回数据：

```
{
  "success": true,
  "msg": "保存成功",
  "data": {
    "caseId": 70003
  }
}
```

3.10. 删除题目代码用例

请求路径	/api/questions/cases/{caseId}
请求方式	DELETE
请求参数	无

> 需要登录：是（JWT）

> 权限要求：TEACHER / ADMIN

请求参数：

参数名	类型	是否必须	备注
caseId	long(path)	是	用例ID

返回数据：

```
{
  "success": true,
  "msg": "删除成功",
  "data": null
}
```

4. 试卷（Paper）管理

4.1. 创建试卷

请求路径	/api/papers
请求方式	POST
请求参数	application/json

> 需要登录：是（JWT）

> 权限要求：TEACHER / ADMIN

请求参数：

参数名	类型	是否必须	备注
paperTitle	string	是	试卷名称
paperDesc	string	否	试卷描述
paperType	int	否	1=manual,2=rule_generated（默认1）
ruleJson	object	否	规则组卷配置（paperType=2 时使用）
status	int	否	1=draft,2=published（默认1）

返回数据：

```
{
  "success": true,
  "msg": "创建成功",
  "data": {
    "paperId": 80001
  }
}
```

4.2. 更新试卷

请求路径	/api/papers/{paperId}
请求方式	PUT
请求参数	application/json

> 需要登录：是（JWT）

> 权限要求：TEACHER / ADMIN

请求参数：

参数名	类型	是否必须	备注
paperId	long(path)	是	试卷ID
paperTitle	string	否	试卷名称
paperDesc	string	否	试卷描述
paperType	int	否	1=manual,2=rule_generated

参数名	类型	是否必须	备注
ruleJson	object	否	规则组卷配置
status	int	否	1=draft,2=published,3=archived

返回数据：

```
{
  "success": true,
  "msg": "更新成功",
  "data": null
}
```

4.3. 删除试卷（软删除）

请求路径	/api/papers/{paperId}
请求方式	DELETE
请求参数	无

> 需要登录：是（JWT）

> 权限要求：TEACHER / ADMIN

请求参数：

参数名	类型	是否必须	备注
paperId	long(path)	是	试卷ID

返回数据：

```
{
  "success": true,
  "msg": "删除成功",
  "data": null
}
```

4.4. 试卷列表（分页）

请求路径	/api/papers
请求方式	GET
请求参数	query

> 需要登录：是（JWT）

> 权限要求：TEACHER / ADMIN

请求参数：

参数名	类型	是否必须	备注
keyword	string	否	按标题搜索
status	int	否	1=draft,2=published,3=archived
page	int	否	页码（默认1）
size	int	否	每页条数（默认10）

返回数据：

```
{
  "success": true,
  "msg": "ok",
  "data": {
    "page": 1,
    "size": 10,
    "total": 1,
    "list": [
      {
        "id": 80001,
        "paperTitle": "指针专题练习",
        "paperType": 1,
        "totalScore": 100,
        "status": 1
      }
    ]
  }
}
```

4.5. 试卷详情（含题目列表与分值）

请求路径	/api/papers/{paperId}
请求方式	GET
请求参数	无

> 需要登录：是（JWT）

> 权限要求：TEACHER / ADMIN

请求参数：

参数名	类型	是否必须	备注
paperId	long(path)	是	试卷ID

返回数据：


```json
{
  "success": true,
  "msg": "ok",
  "data": {
    "id": 80001,
    "paperTitle": "指针专题练习",
    "paperDesc": "…",
    "paperType": 1,
    "totalScore": 100,
    "status": 1,
    "questions": [
      {
        "paperQuestionId": 81001,
        "questionId": 50001,
        "orderNo": 1,
        "score": 10
      },
      {
        "paperQuestionId": 81002,
        "questionId": 50002,
        "orderNo": 2,
        "score": 10
```

```
      }
    ]
  }
}
```

4.6. 试卷添加题目

请求路径	/api/papers/{paperId}/questions
请求方式	POST
请求参数	application/json

> 需要登录：是（JWT）

> 权限要求：TEACHER / ADMIN

> 说明：写快照时，后端把题干/选项/题型/难度等序列化到 qb_paper_question.snapshot_json。

请求参数：

参数名	类型	是否必须	备注
paperId	long(path)	是	试卷ID
questionId	long	是	题目ID
orderNo	int	是	题序（从1开始）
score	int	是	该题分值
snapshot	boolean	否	是否写入题目快照（默认true）

返回数据：

```
{
  "success": true,
  "msg": "添加成功",
  "data": {
    "paperQuestionId": 81003
  }
}
```

4.7. 更新试卷题目（题序/分值）

请求路径	/api/papers/questions/{paperQuestionId}
请求方式	PUT
请求参数	application/json

> 需要登录：是（JWT）

> 权限要求：TEACHER / ADMIN

请求参数：

参数名	类型	是否必须	备注
paperQuestionId	long(path)	是	试卷题目关联ID
orderNo	int	否	题序
score	int	否	分值

返回数据：

```
{
  "success": true,
  "msg": "更新成功",
  "data": null
}
```

4.8. 试卷移除题目

请求路径	/api/papers/questions/{paperQuestionId}
请求方式	DELETE
请求参数	无

> 需要登录：是（JWT）

> 权限要求：TEACHER / ADMIN

请求参数：

参数名	类型	是否必须	备注
paperQuestionId	long(path)	是	试卷题目关联ID

返回数据：

```
{
  "success": true,
  "msg": "移除成功",
  "data": null
}
```

4.9. 重新计算试卷总分

请求路径	/api/papers/{paperId}/recalculate
请求方式	POST
请求参数	无

> 需要登录：是（JWT）

> 权限要求：TEACHER / ADMIN

请求参数：

参数名	类型	是否必须	备注
paperId	long(path)	是	试卷ID

返回数据：

```
{
  "success": true,
  "msg": "ok",
  "data": {
    "paperId": 80001,
    "totalScore": 100
  }
}
```

5. 作业/测试发布（Assignment）

5.1. 创建作业/测试

请求路径	/api/assignments
请求方式	POST
请求参数	application/json

> 需要登录：是（JWT）

> 权限要求：TEACHER / ADMIN

请求参数：

参数名	类型	是否必须	备注
paperId	long	是	关联试卷ID
assignmentTitle	string	是	作业标题
assignmentDesc	string	否	说明
startTime	string(datetime)	否	开始时间（可空，表示立即可做）
endTime	string(datetime)	是	截止时间
timeLimitMin	int	否	限时（分钟，0=不限时）
maxAttempts	int	否	最大作答次数（默认1）
shuffleQuestions	boolean	否	是否打乱题目顺序（默认false）
shuffleOptions	boolean	否	是否打乱选项顺序（默认false）

返回数据：

```json
{
  "success": true,
  "msg": "创建成功",
  "data": {
    "assignmentId": 82001
  }
}
```

5.2. 更新作业/测试

请求路径	/api/assignments/{assignmentId}
请求方式	PUT
请求参数	application/json

> 需要登录：是（JWT）

> 权限要求：TEACHER / ADMIN

请求参数：

参数名	类型	是否必须	备注
assignmentId	long(path)	是	作业ID
assignmentTitle	string	否	作业标题
assignmentDesc	string	否	说明
startTime	string(datetime)	否	开始时间
endTime	string(datetime)	否	截止时间
timeLimitMin	int	否	限时（分钟）
maxAttempts	int	否	最大作答次数
shuffleQuestions	boolean	否	是否打乱题目顺序
shuffleOptions	boolean	否	是否打乱选项顺序

返回数据：

```
{
  "success": true,
  "msg": "更新成功",
  "data": null
}
```

5.3. 发布作业（draft -> published）

请求路径	/api/assignments/{assignmentId}/publish
请求方式	POST
请求参数	无

> 需要登录：是（JWT）

> 权限要求：TEACHER / ADMIN

请求参数：

参数名	类型	是否必须	备注
assignmentId	long(path)	是	作业ID

返回数据：

```
{
  "success": true,
  "msg": "发布成功",
  "data": null
}
```

5.4. 关闭作业（published -> closed）

请求路径	/api/assignments/{assignmentId}/close
请求方式	POST
请求参数	无

> 需要登录：是（JWT）

> 权限要求：TEACHER / ADMIN

请求参数：

参数名	类型	是否必须	备注
assignmentId	long(path)	是	作业ID

返回数据：

```
{
  "success": true,
  "msg": "关闭成功",
  "data": null
}
```

5.5. 设置作业发布对象（指定学生）

请求路径	/api/assignments/{assignmentId}/targets
请求方式	PUT
请求参数	application/json

> 需要登录：是（JWT）

> 权限要求：TEACHER / ADMIN

> 说明：若不设置 targets（或置空），表示对所有学生可见；若设置 targets，则仅 targets 内学生可见。

请求参数：

参数名	类型	是否必须	备注
assignmentId	long(path)	是	作业ID
userIds	array	是	学生用户ID数组（全量覆盖）

返回数据：

```
{
  "success": true,
  "msg": "设置成功",
  "data": null
}
```

5.6. 教师端-作业列表（分页）

请求路径	/api/assignments
请求方式	GET
请求参数	query

> 需要登录：是（JWT）

> 权限要求：TEACHER / ADMIN

请求参数：

参数名	类型	是否必须	备注
keyword	string	否	按标题搜索
publishStatus	int	否	1=draft,2=published,3=closed
page	int	否	页码（默认1）
size	int	否	每页条数（默认10）

返回数据：

```
{
  "success": true,
  "msg": "ok",
  "data": {
    "page": 1,
    "size": 10,
    "total": 1,
    "list": [
      {
        "id": 82001,
        "assignmentTitle": "指针周测",
        "paperId": 80001,
        "endTime": "2026-02-01T23:59:59",
        "publishStatus": 2
      }
    ]
  }
}
```

5.7. 学生端-我的作业列表（分页）

请求路径	/api/assignments/my
请求方式	GET
请求参数	query

> 需要登录：是（JWT）

> 权限要求：STUDENT

请求参数：

参数名	类型	是否必须	备注
status	string	否	筛选：ongoing/expired/all
page	int	否	页码（默认1）
size	int	否	每页条数（默认10）

返回数据：

```
{
  "success": true,
  "msg": "ok",
  "data": {
    "page": 1,
    "size": 10,
    "total": 1,
    "list": [
      {
        "assignmentId": 82001,
        "assignmentTitle": "指针周测",
        "startTime": "2026-01-25T00:00:00",
        "endTime": "2026-02-01T23:59:59",
        "timeLimitMin": 60,
        "maxAttempts": 1,
        "myAttemptCount": 0,
        "publishStatus": 2
      }
    ]
```

```
  }
}
```

5.8. 作业详情（学生/教师通用）

请求路径	/api/assignments/{assignmentId}
请求方式	GET
请求参数	无

> 需要登录：是（JWT）

> 权限要求：STUDENT / TEACHER / ADMIN

请求参数：

参数名	类型	是否必须	备注
assignmentId	long(path)	是	作业ID

返回数据：

```
{
  "success": true,
  "msg": "ok",
  "data": {
    "id": 82001,
    "paperId": 80001,
    "assignmentTitle": "指针周测",
    "assignmentDesc": "…",
    "startTime": "2026-01-25T00:00:00",
    "endTime": "2026-02-01T23:59:59",
    "timeLimitMin": 60,
    "maxAttempts": 1,
    "shuffleQuestions": false,
    "shuffleOptions": false,
    "publishStatus": 2
  }
}
```

6. 作答与练习（Attempt / Answer）

6.1. 开始作业作答（创建 Attempt，并生成题单快照）

请求路径	/api/attempts/assignment/{assignmentId}/start
请求方式	POST
请求参数	application/json

> 需要登录：是（JWT）

> 权限要求：STUDENT

> 说明：后端会校验：是否在开始/截止时间内、是否超过 maxAttempts。生成 qb_attempt_question + qb_answer（空草稿）。

请求参数：

参数名	类型	是否必须	备注
assignmentId	long(path)	是	作业ID

返回数据：

```
{
  "success": true,
  "msg": "开始成功",
  "data": {
    "attemptId": 90001,
    "attemptType": 1,
    "status": 1,
    "startedAt": "2026-01-30T10:00:00",
    "timeLimitMin": 60,
    "questionCount": 10
  }
}
```

6.2. 开始练习（随机/自适应）

请求路径	/api/attempts/practice/start
请求方式	POST
请求参数	application/json

> 需要登录：是（JWT）

> 权限要求：STUDENT

> 说明：mode=adaptive 时，后端会基于 tagMastery/ability 计算权重抽题，并写入 qb_practice_generation.debug_json 方便论文展示。

请求参数：

参数名	类型	是否必须	备注
mode	string	是	random / adaptive
scope.tagIds	array	否	范围：标签ID数组
scope.chapters	array	否	范围：章节数组
scope.questionTypes	array	否	范围：题型数组
totalScore	int	否	总分（默认100）
ruleId	long	否	使用指定练习规则（可选）

返回数据：

```json
{
  "success": true,
  "msg": "开始成功",
  "data": {
    "attemptId": 90002,
    "attemptType": 2,
    "status": 1,
    "startedAt": "2026-01-30T11:00:00",
    "questionCount": 10,
    "practiceGenerationId": 99001
  }
}
```

6.3. 获取作答题单（作答页用：题干/选项快照，不返回标准答案）

请求路径	/api/attempts/{attemptId}/questions
请求方式	GET
请求参数	无

> 需要登录：是（JWT）

> 权限要求：STUDENT

> 说明：为了防止作答时泄露答案，作答中的 snapshot 不应包含 standardAnswer/analysis。提交后再通过 result 接口返回解析。

请求参数：

参数名	类型	是否必须	备注
attemptId	long(path)	是	Attempt ID

返回数据：

```
{
  "success": true,
  "msg": "ok",
  "data": [
    {
      "attemptQuestionId": 91001,
      "orderNo": 1,
      "questionId": 50001,
      "score": 10,
      "questionType": 1,
      "difficulty": 3,
      "snapshot": {
        "title": "指针与数组",
        "stem": "以下代码输出是什么？ ...",
        "options": [
          {
            "label": "A",
            "content": "xxx"
          },
```

```
          {
            "label": "B",
            "content": "yyy"
          }
        ]
      },
      "answer": {
        "answerId": 92001,
        "answerStatus": 1,
        "answerContent": "B"
      }
    }
  ]
}
```


6.4. 保存单题草稿（不提交）

请求路径	/api/answers/{answerId}/draft
请求方式	PUT
请求参数	application/json

> 需要登录：是（JWT）

> 权限要求：STUDENT

请求参数：

参数名	类型	是否必须	备注
answerId	long(path)	是	答案ID
answerContent	string	是	学生答案（文本/JSON 字符串）
answerFormat	int	否	1=text,2=json（默认1）

返回数据：


```
{
  "success": true,
  "msg": "保存成功",
```

```
  "data": null
}
```

6.5. 提交单题答案（可选：如果你做“逐题提交”）

请求路径	/api/answers/{answerId}/submit
请求方式	PUT
请求参数	application/json

> 需要登录：是（JWT）

> 权限要求：STUDENT

> 说明：如果你只做“整卷提交”，可以不实现该接口，全部在 6.6 提交 Attempt 时一次性提交。

请求参数：

参数名	类型	是否必须	备注
answerId	long(path)	是	答案ID
answerContent	string	是	学生答案
answerFormat	int	否	1=text,2=json（默认1）

返回数据：

```
{
  "success": true,
  "msg": "提交成功",
  "data": null
}
```

6.6. 提交整次作答（触发判分/LLM评分/统计更新）

请求路径	/api/attempts/{attemptId}/submit
请求方式	POST
请求参数	application/json

> 需要登录：是（JWT）

> 权限要求：STUDENT

> 说明：后端流程：锁定 attempt→客观题规则判分→主观题调用 LLM 输出 JSON（score/deductions/reason/confidence）→置信度低或解析失败则 needs_review。

请求参数：

参数名	类型	是否必须	备注
attemptId	long(path)	是	Attempt ID
durationSec	int	否	作答耗时（秒，可选，后端也可自行计算）

返回数据：

```
{
  "success": true,
  "msg": "提交成功",
  "data": {
    "attemptId": 90001,
    "status": 3,
    "objectiveScore": 60,
    "subjectiveScore": 0,
    "totalScore": 60,
    "needsReview": true
  }
}
```

6.7. 查看作答结果（含解析、每题得分、LLM/人工记录）

请求路径	/api/attempts/{attemptId}/result
请求方式	GET
请求参数	无

> 需要登录：是（JWT）

> 权限要求：STUDENT / TEACHER / ADMIN

> 说明：学生端能看到最终解析与得分；教师端可额外看到更多字段（例如 rubric/证据链）。

请求参数：

参数名	类型	是否必须	备注
attemptId	long(path)	是	Attempt ID

返回数据：

```
{
  "success": true,
  "msg": "ok",
  "data": {
    "attempt": {
      "attemptId": 90001,
      "attemptType": 1,
      "status": 4,
      "totalScore": 85,
      "objectiveScore": 60,
      "subjectiveScore": 25,
      "needsReview": false
    },
    "items": [
      {
        "orderNo": 1,
        "questionId": 50001,
        "questionType": 1,
        "score": 10,
        "myAnswer": "B",
        "finalScore": 10,
        "isCorrect": true,
        "analysisText": "解析：..."
      },
      {
        "orderNo": 2,
        "questionId": 50010,
        "questionType": 5,
        "score": 25,
        "myAnswer": "……",
        "finalScore": 20,
```

```
        "grading": {
          "finalMode": 3,
          "records": [
            {
              "gradingMode": 2,
              "score": 18,
              "confidence": 0.62,
              "llmCallId": 99011,
              "needsReview": true
            },
            {
              "gradingMode": 3,
              "score": 20,
              "reviewerId": 20001,
              "comment": "表达不完整扣5分",
              "isFinal": true
            }
          ]
        }
      }
    ]
  }
}
```

6.8. 我的作答记录（分页）

请求路径	/api/attempts/my
请求方式	GET
请求参数	query

> 需要登录：是（JWT）

> 权限要求：STUDENT

请求参数：

参数名	类型	是否必须	备注
attemptType	int	否	1=assignment,2=practice
page	int	否	页码（默认1）

参数名	类型	是否必须	备注
size	int	否	每页条数（默认10）

返回数据：

```
{
  "success": true,
  "msg": "ok",
  "data": {
    "page": 1,
    "size": 10,
    "total": 1,
    "list": [
      {
        "attemptId": 90001,
        "attemptType": 1,
        "status": 4,
        "totalScore": 85,
        "createdAt": "2026-01-30T10:00:00"
      }
    ]
  }
}
```

7. 错题集与统计（掌握度/能力值/做题统计）

7.1. 我的错题集（分页）

请求路径	/api/stats/wrong-questions
请求方式	GET
请求参数	query

> 需要登录：是（JWT）

> 权限要求：STUDENT

请求参数：

参数名	类型	是否必须	备注
tagId	long	否	按标签筛选（可选）
chapter	string	否	按章节筛选（可选）
isResolved	boolean	否	是否已解决
page	int	否	页码（默认1）
size	int	否	每页条数（默认10）

返回数据：

```
{
  "success": true,
  "msg": "ok",
  "data": {
    "page": 1,
    "size": 10,
    "total": 1,
    "list": [
      {
        "questionId": 50001,
        "wrongCount": 2,
        "lastWrongAt": "2026-01-20T12:00:00",
        "isResolved": false
      }
    ]
  }
}
```

7.2. 错题标记已解决

请求路径	/api/stats/wrong-questions/{questionId}/resolve
请求方式	POST
请求参数	无

> 需要登录：是（JWT）

> 权限要求：STUDENT

请求参数：

参数名	类型	是否必须	备注
questionId	long(path)	是	题目ID

返回数据：

```
{
  "success": true,
  "msg": "已标记",
  "data": null
}
```

7.3. 获取我的知识点掌握度（TagMastery）

请求路径	/api/stats/mastery
请求方式	GET
请求参数	query

> 需要登录：是（JWT）

> 权限要求：STUDENT

请求参数：

参数名	类型	是否必须	备注
tagType	int	否	1=knowledge,2=chapter,3=custom（默认1）

返回数据：

```
{
  "success": true,
  "msg": "ok",
  "data": [
    {
      "tagId": 1,
```

```
      "tagName": "指针",
      "masteryValue": 0.63,
      "attemptCount": 12,
      "correctCount": 8
    },
    {
      "tagId": 2,
      "tagName": "数组",
      "masteryValue": 0.4,
      "attemptCount": 5,
      "correctCount": 2
    }
  ]
}
```

7.4. 获取我的能力值（UserAility）

请求路径	/api/stats/ability
请求方式	GET
请求参数	无

> 需要登录：是（JWT）

> 权限要求：STUDENT

请求参数：

无

返回数据：

```
{
  "success": true,
  "msg": "ok",
  "data": {
    "abilityScore": 72,
    "updatedAt": "2026-01-30T10:05:00"
  }
}
```

7.5. 获取我的单题统计（QuestionUserStat，分页）

请求路径	/api/stats/question-stats
请求方式	GET
请求参数	query

> 需要登录：是（JWT）

> 权限要求：STUDENT

请求参数：

参数名	类型	是否必须	备注
questionId	long	否	按题目ID筛选（可选）
page	int	否	页码（默认1）
size	int	否	每页条数（默认10）

返回数据：

```
{
  "success": true,
  "msg": "ok",
  "data": {
    "page": 1,
    "size": 10,
    "total": 1,
    "list": [
      {
        "questionId": 50001,
        "attemptCount": 3,
        "correctCount": 2,
        "lastAttemptAt": "2026-01-30T10:00:00"
      }
    ]
  }
}
```

8. 批改与复核（教师端）

8.1. 教师-待复核答案列表（分页）

请求路径	/api/teacher/review/answers
请求方式	GET
请求参数	query

> 需要登录：是（JWT）

> 权限要求：TEACHER / ADMIN

请求参数：

参数名	类型	是否必须	备注
assignmentId	long	否	按作业筛选（可选）
needsReview	boolean	否	是否需要复核（默认true）
page	int	否	页码（默认1）
size	int	否	每页条数（默认10）

返回数据：

```
{
  "success": true,
  "msg": "ok",
  "data": {
    "page": 1,
    "size": 10,
    "total": 1,
    "list": [
      {
        "answerId": 92010,
        "attemptId": 90001,
        "studentId": 10001,
        "questionId": 50010,
        "questionType": 5,
        "score": 25,
        "currentFinalScore": 0,
        "needsReview": true
      }
```

```
      ]
    }
  }
```

8.2. 教师-查看答案详情与评分证据（含 LLM 原始输出）

请求路径	/api/teacher/answers/{answerId}/evidence
请求方式	GET
请求参数	无

> 需要登录：是（JWT）

> 权限要求：TEACHER / ADMIN

> 说明：该接口用于“教师看证据链”：题目快照/评分细则/学生答案/LLM输出/解析结果。

请求参数：

参数名	类型	是否必须	备注
answerId	long(path)	是	答案ID

返回数据：

```
{
  "success": true,
  "msg": "ok",
  "data": {
    "answerId": 92010,
    "student": {
      "id": 10001,
      "displayName": "张三"
    },
    "questionSnapshot": {
      "questionId": 50010,
      "stem": "简答：解释指针与数组的区别…",
      "rubric": [
        {
          "point": "说明指针存地址",
          "score": 10
```

```
        },
        {
          "point": "说明数组是连续内存",
          "score": 10
        },
        {
          "point": "举例说明",
          "score": 5
        }
      ]
    },
    "studentAnswer": "……",
    "gradingRecords": [
      {
        "gradingMode": 2,
        "score": 18,
        "confidence": 0.62,
        "needsReview": true,
        "llmCall": {
          "llmCallId": 99011,
          "modelName": "qwen-plus",
          "promptText": "（省略）",
          "responseText": "（省略）",
          "responseJson": {
            "score": 18,
            "deductions": [],
            "reason": "……",
            "confidence": 0.62
          },
          "callStatus": 1
        }
      }
    ]
  }
}
```


8.3. 教师-人工给分/改分（最终结果）

请求路径	/api/teacher/answers/{answerId}/grade
请求方式	POST
请求参数	application/json

> 需要登录：是（JWT）

> 权限要求：TEACHER / ADMIN

> 说明：后端写入 qb_grading_record(mode=MANUAL,is_final=true) 并更新 qb_answer.final_score。

请求参数：

参数名	类型	是否必须	备注
answerId	long(path)	是	答案ID
score	int	是	最终得分（0~题目分值）
comment	string	否	评语/扣分说明

返回数据：

```
{
  "success": true,
  "msg": "已提交人工评分",
  "data": null
}
```

8.4. 教师-重新触发 LLM 评分（重试）

请求路径	/api/teacher/answers/{answerId}/llm-retry
请求方式	POST
请求参数	application/json

> 需要登录：是（JWT）

> 权限要求：TEACHER / ADMIN

> 说明：如果你实现“一致性校验”，可多次调用并对比分数差异；差异过大则 needs_review=true。

请求参数：

参数名	类型	是否必须	备注
answerId	long(path)	是	答案ID
modelName	string	否	默认 qwen-plus
temperature	float	否	默认0.2
times	int	否	调用次数（用于一致性校验，可选，默认1）

返回数据：

```
{
  "success": true,
  "msg": "已触发",
  "data": {
    "llmCallIds": [
      99021,
      99022
    ]
  }
}
```

8.5. 教师-作业成绩列表（分页）

请求路径	/api/teacher/assignments/{assignmentId}/scores
请求方式	GET
请求参数	query

> 需要登录：是（JWT）

> 权限要求：TEACHER / ADMIN

请求参数：

参数名	类型	是否必须	备注
assignmentId	long(path)	是	作业ID
page	int	否	页码（默认1）
size	int	否	每页条数（默认10）

返回数据：

```
{
  "success": true,
  "msg": "ok",
  "data": {
    "page": 1,
    "size": 10,
    "total": 1,
    "list": [
      {
        "studentId": 10001,
        "displayName": "张三",
        "attemptId": 90001,
        "totalScore": 85,
        "needsReview": false,
        "submittedAt": "2026-01-30T10:30:00"
      }
    ]
  }
}
```

9. 申诉复核（Appeal）

9.1. 学生提交申诉

请求路径	/api/appeals
请求方式	POST
请求参数	application/json

> 需要登录：是（JWT）

> 权限要求：STUDENT

> 说明：数据库 qb_appeal 中未建附件表，attachments 可存成 JSON 字符串或走文件服务（看你实现）。

请求参数：

参数名	类型	是否必须	备注
answerId	long	是	要申诉的答案ID
reasonText	string	是	申诉理由
attachments	array	否	附件URL列表（可选）

返回数据：

```
{
  "success": true,
  "msg": "提交成功",
  "data": {
    "appealId": 60001
  }
}
```

9.2. 学生查看我的申诉列表（分页）

请求路径	/api/appeals/my
请求方式	GET
请求参数	query

> 需要登录：是（JWT）

> 权限要求：STUDENT

请求参数：

参数名	类型	是否必须	备注
status	int	否	1=pending,2=approved,3=rejected,4=resolved
page	int	否	页码（默认1）
size	int	否	每页条数（默认10）

返回数据：

```
{
  "success": true,
  "msg": "ok",
  "data": {
    "page": 1,
    "size": 10,
    "total": 1,
    "list": [
      {
        "appealId": 60001,
        "answerId": 92010,
        "appealStatus": 1,
        "createdAt": "2026-01-30T12:00:00"
      }
    ]
  }
}
```

9.3. 教师查看待处理申诉（分页）

请求路径	/api/teacher/appeals
请求方式	GET
请求参数	query

> 需要登录：是（JWT）

> 权限要求：TEACHER / ADMIN

请求参数：

参数名	类型	是否必须	备注
status	int	否	1=pending,2=approved,3=rejected,4=resolved（默认1）
page	int	否	页码（默认1）
size	int	否	每页条数（默认10）

返回数据：

```json
{
  "success": true,
  "msg": "ok",
  "data": {
    "page": 1,
    "size": 10,
    "total": 1,
    "list": [
      {
        "appealId": 60001,
        "answerId": 92010,
        "studentId": 10001,
        "reasonText": "我认为应该得满分…",
        "appealStatus": 1
      }
    ]
  }
}
```

9.4. 教师处理申诉（给最终结论/分数）

请求路径	/api/teacher/appeals/{appealId}/handle
请求方式	POST
请求参数	application/json

> 需要登录：是（JWT）

> 权限要求：TEACHER / ADMIN

> 说明：若 finalScore 发生变化：写入 MANUAL 评分记录并更新答案 final_score，同时把申诉状态置为 resolved。

请求参数：

参数名	类型	是否必须	备注
appealId	long(path)	是	申诉ID
action	string	是	approve / reject
finalScore	int	否	最终分（approve 时可给）

参数名	类型	是否必须	备注
decisionComment	string	否	处理说明

返回数据：

```
{
  "success": true,
  "msg": "已处理",
  "data": null
}
```

10. 答疑区（Forum）

10.1. 发帖

请求路径	/api/posts
请求方式	POST
请求参数	application/json

> 需要登录：是（JWT）

> 权限要求：STUDENT / TEACHER / ADMIN

请求参数：

参数名	类型	是否必须	备注
title	string	是	帖子标题
content	string	是	帖子内容
tagIds	array	否	关联标签（可选，复用 qb_tag）
relatedQuestionId	long	否	关联题目ID（可选）

返回数据：

```
{
  "success": true,
  "msg": "发布成功",
```

```
  "data": {
    "postId": 70001
  }
}
```

10.2. 帖子列表（分页）

请求路径	/api/posts
请求方式	GET
请求参数	query

> 需要登录：是（JWT）

> 权限要求：STUDENT / TEACHER / ADMIN

请求参数：

参数名	类型	是否必须	备注
tagId	long	否	按标签筛选
keyword	string	否	标题关键字
page	int	否	页码（默认1）
size	int	否	每页条数（默认10）

返回数据：

```
{
  "success": true,
  "msg": "ok",
  "data": {
    "page": 1,
    "size": 10,
    "total": 1,
    "list": [
      {
        "postId": 70001,
        "title": "指针题求解",
        "authorId": 10001,
        "createdAt": "2026-01-30T12:10:00"
```

```
      }
    ]
  }
}
```

10.3. 帖子详情（含回复）

请求路径	/api/posts/{postId}
请求方式	GET
请求参数	无

> 需要登录：是（JWT）

> 权限要求：STUDENT / TEACHER / ADMIN

请求参数：

参数名	类型	是否必须	备注
postId	long(path)	是	帖子ID

返回数据：

```
{
  "success": true,
  "msg": "ok",
  "data": {
    "post": {
      "postId": 70001,
      "title": "指针题求解",
      "content": "……",
      "authorId": 10001
    },
    "replies": [
      {
        "replyId": 71001,
        "authorId": 20001,
        "content": "你这里应该注意…",
        "parentReplyId": null
      }
```

```
      ]
    }
  }
```

10.4. 回帖/回复（支持楼中楼）

请求路径	/api/posts/{postId}/replies
请求方式	POST
请求参数	application/json

> 需要登录：是（JWT）

> 权限要求：STUDENT / TEACHER / ADMIN

请求参数：

参数名	类型	是否必须	备注
postId	long(path)	是	帖子ID
content	string	是	回复内容
parentReplyId	long	否	父回复ID（楼中楼，可选）

返回数据：

```json
{
  "success": true,
  "msg": "回复成功",
  "data": {
    "replyId": 71002
  }
}
```

10.5. 删除帖子（软删除）

请求路径	/api/posts/{postId}
请求方式	DELETE
请求参数	无

> 需要登录：是（JWT）

> 权限要求：STUDENT / TEACHER / ADMIN

> 说明：通常只允许作者删除；教师/管理员可强制删除。

请求参数：

参数名	类型	是否必须	备注
postId	long(path)	是	帖子ID

返回数据：

```
{
  "success": true,
  "msg": "删除成功",
  "data": null
}
```

10.6. 删除回复（软删除）

请求路径	/api/replies/{replyId}
请求方式	DELETE
请求参数	无

> 需要登录：是（JWT）

> 权限要求：STUDENT / TEACHER / ADMIN

请求参数：

参数名	类型	是否必须	备注
replyId	long(path)	是	回复ID

返回数据：

```
{
  "success": true,
  "msg": "删除成功",
  "data": null
}
```

10.7. 关闭帖子（禁止继续回复）

请求路径	/api/posts/{postId}/close
请求方式	POST
请求参数	无

> 需要登录：是（JWT）

> 权限要求：TEACHER / ADMIN

请求参数：

参数名	类型	是否必须	备注
postId	long(path)	是	帖子ID

返回数据：

```
{
  "success": true,
  "msg": "已关闭",
  "data": null
}
```

11. LLM 调用记录（证据链）

11.1. 查询 LLM 调用记录列表（按业务关联）

请求路径	/api/llm/calls
请求方式	GET
请求参数	query

> 需要登录：是（JWT）

> 权限要求：TEACHER / ADMIN

请求参数：

参数名	类型	是否必须	备注
bizType	int	否	1=QUESTION_ANALYSIS,2=SUBJECTIVE_GRADING,3=OTHI
bizId	long	否	业务ID（如 questionId/answerId）
page	int	否	页码（默认1）
size	int	否	每页条数（默认10）

返回数据：

```
{
  "success": true,
  "msg": "ok",
  "data": {
    "page": 1,
    "size": 10,
    "total": 1,
    "list": [
      {
        "llmCallId": 99011,
        "bizType": 2,
        "bizId": 92010,
        "modelName": "qwen-plus",
        "callStatus": 1,
        "latencyMs": 1200,
        "createdAt": "2026-01-30T10:31:00"
      }
    ]
  }
}
```

11.2. 获取 LLM 调用详情

请求路径	/api/llm/calls/{llmCallId}
请求方式	GET
请求参数	无

> 需要登录：是（JWT）

> 权限要求：TEACHER / ADMIN

请求参数：

参数名	类型	是否必须	备注
llmCallId	long(path)	是	LLM 调用ID

返回数据：

```
{
  "success": true,
  "msg": "ok",
  "data": {
    "llmCallId": 99011,
    "bizType": 2,
    "bizId": 92010,
    "modelName": "qwen-plus",
    "promptText": "（省略）",
    "responseText": "（省略）",
    "responseJson": {
      "score": 18,
      "deductions": [],
      "reason": "……",
      "confidence": 0.62
    },
    "callStatus": 1,
    "tokensPrompt": 800,
    "tokensCompletion": 300,
    "latencyMs": 1200
  }
}
```

12. 千问（Qwen）调用说明（后端集成参数）

本项目的 LLM 调用使用 **通义千问 OpenAI 兼容模式**：

- `base_url`： `https://dashscope.aliyuncs.com/compatible-mode/v1`
- Chat Completions： `POST https://dashscope.aliyuncs.com/compatible-mode/v1/chat/completions`
- 鉴权：HTTP Header `Authorization: Bearer $DASHSCOPE_API_KEY`

12.1 推荐的请求体（示例）

```
{
  "model": "qwen-plus",
  "messages": [
    {"role": "system", "content": "You are a helpful assistant."},
    {"role": "user", "content": "..."}
  ],
  "temperature": 0.2,
  "response_format": {
    "type": "json_object"
  }
}
```

12.2 结构化输出（用于“主观题评分 JSON”）

- 千问支持 `response_format.type = json_object` 或 `json_schema`，用于约束模型输出 JSON。
- 若使用 `json_object`，提示词中**必须显式要求输出 JSON**（否则会报错/输出不稳定）。

评分 JSON 建议统一为：

```
{
  "score": 18,
  "deductions": [
    {"point": "遗漏关键点：数组连续内存", "deduct": 5, "reason": "没有说明数组是连续存储"}
  ],
  "reason": "总体理解正确，但解释不完整",
  "confidence": 0.62
}
```

你开题报告里强调的“置信度阈值 + 分歧触发复核 + 申诉证据链”，在接口与表设计上分别对应：

- 阈值触发：`qb_grading_record.needs_review` + `qb_attempt.needs_review`
- 证据链：`qb_llm_call`（prompt/response/json）+ `qb_grading_record.llm_call_id`

13. 系统日志（管理员）

13.1. 审计日志（谁改了什么，分页）

请求路径	**/api/admin/audit-logs**
请求方式	GET
请求参数	query

> 需要登录：是（JWT）

> 权限要求：ADMIN

请求参数：

参数名	类型	是否必须	备注
userId	long	否	操作人ID
action	string	否	动作，如 CREATE/UPDATE/DELETE/PUBLISH
entityType	string	否	实体类型，如 QUESTION/TAG/PAPER/ASSIGNMENT
entityId	long	否	实体ID
startTime	string(datetime)	否	开始时间
endTime	string(datetime)	否	结束时间
page	int	否	页码（默认1）
size	int	否	每页条数（默认10）

返回数据：

```json
{
  "success": true,
  "msg": "ok",
  "data": {
    "page": 1,
    "size": 10,
    "total": 1,
    "list": [
      {
        "logId": 1,
        "userId": 20001,
        "action": "UPDATE",
        "entityType": "QUESTION",
        "entityId": 50001,
        "createdAt": "2026-01-30T09:00:00"
      }
    ]
  }
}
```

13.2. 登录日志（分页）

请求路径	**/api/admin/login-logs**
请求方式	GET
请求参数	query

> 需要登录：是（JWT）

> 权限要求：ADMIN

请求参数：

参数名	**类型**	**是否必须**	**备注**
username	string	否	用户名
successFlag	boolean	否	是否成功
startTime	string(datetime)	否	开始时间
endTime	string(datetime)	否	结束时间
page	int	否	页码（默认1）

参数名	类型	是否必须	备注
size	int	否	每页条数（默认10）

返回数据：

```
{
  "success": true,
  "msg": "ok",
  "data": {
    "page": 1,
    "size": 10,
    "total": 1,
    "list": [
      {
        "logId": 1,
        "username": "student01",
        "successFlag": true,
        "ipAddr": "127.0.0.1",
        "loginAt": "2026-01-30T08:00:00"
      }
    ]
  }
}
```

附录 A：diagram.md 中整理的 UML/流程图（原样收录，便于你论文直接引用）

> 说明：以下 PlantUML 代码来自你提供的 `diagram.md`。

```
用例图
@startuml
left to right direction
actor 学生 as Student
actor 教师 as Teacher
actor 管理员 as Admin

rectangle "题库管理系统" {
  usecase "登录/退出" as UC_Login

  usecase "浏览题库\n(标签/章节/难度检索)" as UC_Browse
  usecase "开始练习\n(随机/自适应)" as UC_Practice
```

```
  usecase "提交答案" as UC_Submit
  usecase "查看结果与解析" as UC_View
  usecase "错题集/练习记录" as UC_Wrong

  usecase "标签管理" as UC_Tag
  usecase "题目管理\n(CRUD)" as UC_Question
  usecase "组卷" as UC_Paper
  usecase "发布作业/测试" as UC_Publish
  usecase "查看成绩/批改记录" as UC_Grade

  usecase "LLM辅助评分\n(主观题)" as UC_LLM
  usecase "申诉复核" as UC_Appeal

  usecase "答疑区\n发帖/回帖" as UC_Forum
}

Student --> UC_Login
Student --> UC_Browse
Student --> UC_Practice
Student --> UC_Submit
Student --> UC_View
Student --> UC_Wrong
Student --> UC_Appeal
Student --> UC_Forum

Teacher --> UC_Login
Teacher --> UC_Tag
Teacher --> UC_Question
Teacher --> UC_Paper
Teacher --> UC_Publish
Teacher --> UC_Grade
Teacher --> UC_LLM
Teacher --> UC_Appeal
Teacher --> UC_Forum

Admin --> UC_Login
@enduml
```

四个流程图

1:

```
@startuml
|Teacher|
```

```
start
:Login;
:Open "Question Management";
|System|
:Verify JWT & Role(TEACHER);
|Teacher|
:Create Question (type/difficulty/chapter/stem/options/answer);
:Bind Tags;
:Save Draft;
if (Generate analysis by LLM?) then (Yes)
  |System|
  :Build prompt (stem + standard_answer + rubric for analysis);
  :Create LLMCall(biz_type=QUESTION_ANALYSIS);
  :Invoke LLM;
  if (Response OK?) then (Yes)
    :Store llm_response & analysis_draft;
  else (No)
    :Mark analysis_source = MANUAL_PENDING;
  endif
  |Teacher|
  :Review/Edit analysis;
endif
:Publish Question;
|System|
:Set question.status = PUBLISHED;
:Write audit log;
stop
@enduml
```

2:

```
@startuml
|Student|
start
:Login;
:Choose "Practice" or "Assignment";
|System|
:Verify JWT & Role(STUDENT);
if (Entry = Assignment?) then (Yes)
  :Check deadline & remaining attempts;
  if (Allowed?) then (Yes)
    :Create Attempt(type=ASSIGNMENT);
    :Generate AttemptQuestion list from Paper (snapshot/order/score);
  else (No)
    :Reject with reason (expired / no attempts left);
```

```
    stop
  endif
else (Practice)
  :Create Attempt(type=PRACTICE);
  :Generate AttemptQuestion list (Random/Adaptive);
endif

|Student|
repeat
  :Answer one question;
  :Save draft(optional);
repeat while (More questions?)

:Submit Attempt;

|System|
:Lock attempt (status=SUBMITTED);
:Auto-grade objective questions;
:Write AUTO GradingRecord(s);
:Compute total_score(objective part);

if (Has subjective questions?) then (Yes)
  :Mark attempt.status = GRADING_PENDING;
else (No)
  :Mark attempt.status = GRADED;
endif

:Update stats (wrong book / question stat / mastery / ability);
|Student|
:View result & analysis;
stop
@enduml
```

3:

```
@startuml
|System|
start
:Detect subjective answers in submitted attempt;
:For each subjective Answer -> build rubric & prompt;
:Create LLMCall(biz_type=SUBJECTIVE_GRADING);
:Invoke LLM;

if (LLM returns valid JSON?) then (Yes)
  :Parse {score, deductions, reason, confidence};
```

```
  if (confidence >= threshold AND consistency OK?) then (Yes)
    :Create GradingRecord(mode=LLM, needs_review=false);
    :Set Answer.final_score = llm_score;
  else (No)
    :Create GradingRecord(mode=LLM, needs_review=true);
    :Set Answer.status = NEED_REVIEW;
  endif
else (No)
  :Create GradingRecord(mode=LLM, needs_review=true, detail="parse
failed");
  :Set Answer.status = NEED_REVIEW;
endif

:If any Answer NEED_REVIEW -> Attempt.needs_review=true;

|Teacher|
:Open "Review Center";
:View evidence (student answer + rubric + LLM raw + parsed JSON);
:Manual adjust score (optional);
:Confirm final score & comment;
|System|
:Create GradingRecord(mode=MANUAL, is_final=true);
:Set Answer.final_score & graded_at;
:Attempt status -> GRADED (when all graded);

|Student|
:View final result;
if (Submit appeal?) then (Yes)
  :Fill appeal reason & attachments;
  |System|
  :Create Appeal(status=PENDING);
  |Teacher|
  :Handle appeal (approve/reject);
  |System|
  :Update Appeal(status=RESOLVED);
  :If score changed -> update Answer.final_score and record MANUAL
grading;
endif
stop
@enduml

4:
@startuml
|Student|
```

```
start
:Login;
:Select scope (tags/chapters);
:Click "Adaptive Practice";

|System|
:Load candidate questions in scope (PUBLISHED);
:Load TagMastery (0~1) and UserAbility (0~100);
:Map ability -> target difficulty range (d-1..d+1);

:For each candidate question compute weight:
- weakness = 1 - mastery(tag)
- difficulty_match = |q.diff - target| small -> higher
- novelty = 1 / (1 + user_attempt_count)
- exclude recently done;

:Apply constraints:
- total_score = 100
- question type distribution (config)
- no duplicate question;

:Weighted random sampling by groups (type/score);
if (Not enough candidates?) then (Yes)
  :Relax difficulty range or report "insufficient coverage";
endif

:Create Attempt(type=PRACTICE);
:Create AttemptQuestion list with snapshot/order/score;
|Student|
:Enter answering page;
stop
@enduml

sql
-- ==========================================
-- Database: question_bank (example)
-- ==========================================

-- -------------------------
-- 0) RBAC
-- -------------------------
CREATE TABLE sys_user (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  username VARCHAR(64) NOT NULL,
```

```
  password_hash VARCHAR(255) NOT NULL,
  display_name VARCHAR(64) DEFAULT NULL,
  email VARCHAR(128) DEFAULT NULL,
  status TINYINT NOT NULL DEFAULT 1 COMMENT '1=active,0=disabled',
  last_login_at DATETIME(3) DEFAULT NULL,
  created_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  updated_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE
CURRENT_TIMESTAMP(3),
  is_deleted TINYINT NOT NULL DEFAULT 0,
  PRIMARY KEY (id),
  UNIQUE KEY uk_sys_user_username (username),
  KEY idx_sys_user_status (status)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE sys_role (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  role_code VARCHAR(64) NOT NULL COMMENT 'e.g. STUDENT/TEACHER/ADMIN',
  role_name VARCHAR(64) NOT NULL,
  created_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  updated_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE
CURRENT_TIMESTAMP(3),
  PRIMARY KEY (id),
  UNIQUE KEY uk_sys_role_code (role_code)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE sys_permission (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  perm_code VARCHAR(128) NOT NULL,
  perm_name VARCHAR(128) NOT NULL,
  resource VARCHAR(255) DEFAULT NULL COMMENT 'api/resource identifier',
  created_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  PRIMARY KEY (id),
  UNIQUE KEY uk_sys_perm_code (perm_code)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE sys_user_role (
  user_id BIGINT UNSIGNED NOT NULL,
  role_id BIGINT UNSIGNED NOT NULL,
  created_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  PRIMARY KEY (user_id, role_id),
  CONSTRAINT fk_user_role_user FOREIGN KEY (user_id) REFERENCES
sys_user(id),
  CONSTRAINT fk_user_role_role FOREIGN KEY (role_id) REFERENCES
sys_role(id)
```

```
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE sys_role_permission (
  role_id BIGINT UNSIGNED NOT NULL,
  perm_id BIGINT UNSIGNED NOT NULL,
  created_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  PRIMARY KEY (role_id, perm_id),
  CONSTRAINT fk_role_perm_role FOREIGN KEY (role_id) REFERENCES
sys_role(id),
  CONSTRAINT fk_role_perm_perm FOREIGN KEY (perm_id) REFERENCES
sys_permission(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- --------------------------
-- 1) Tag / Question
-- --------------------------
CREATE TABLE qb_tag (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  tag_name VARCHAR(128) NOT NULL,
  tag_code VARCHAR(128) DEFAULT NULL,
  parent_id BIGINT UNSIGNED DEFAULT NULL,
  tag_level INT NOT NULL DEFAULT 1,
  tag_type TINYINT NOT NULL DEFAULT 1 COMMENT
'1=knowledge,2=chapter,3=custom',
  sort_order INT NOT NULL DEFAULT 0,
  created_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  updated_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE
CURRENT_TIMESTAMP(3),
  is_deleted TINYINT NOT NULL DEFAULT 0,
  PRIMARY KEY (id),
  UNIQUE KEY uk_qb_tag_name (tag_name),
  KEY idx_qb_tag_parent (parent_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE qb_llm_call (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  biz_type TINYINT NOT NULL COMMENT
'1=QUESTION_ANALYSIS,2=SUBJECTIVE_GRADING,3=OTHER',
  biz_id BIGINT UNSIGNED DEFAULT NULL,
  model_name VARCHAR(64) NOT NULL,
  prompt_text LONGTEXT NOT NULL,
  response_text LONGTEXT DEFAULT NULL,
  response_json JSON DEFAULT NULL,
  call_status TINYINT NOT NULL DEFAULT 0 COMMENT
```

```
'0=pending,1=success,2=failed',
  latency_ms INT DEFAULT NULL,
  tokens_prompt INT DEFAULT NULL,
  tokens_completion INT DEFAULT NULL,
  cost_amount DECIMAL(10,4) DEFAULT NULL,
  created_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  PRIMARY KEY (id),
  KEY idx_llm_call_biz (biz_type, biz_id),
  KEY idx_llm_call_created (created_at)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE qb_question (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  title VARCHAR(255) DEFAULT NULL,
  question_type TINYINT NOT NULL COMMENT
'1=single,2=multiple,3=true_false,4=blank,5=short,6=code,7=code_reading',
  difficulty TINYINT NOT NULL DEFAULT 1 COMMENT '1~5',
  chapter VARCHAR(64) DEFAULT NULL,
  stem LONGTEXT NOT NULL,
  standard_answer LONGTEXT DEFAULT NULL,
  answer_format TINYINT NOT NULL DEFAULT 1 COMMENT '1=text,2=json',
  analysis_text LONGTEXT DEFAULT NULL,
  analysis_source TINYINT NOT NULL DEFAULT 1 COMMENT
'1=manual,2=llm_draft,3=llm_final',
  analysis_llm_call_id BIGINT UNSIGNED DEFAULT NULL,
  status TINYINT NOT NULL DEFAULT 1 COMMENT
'1=draft,2=published,3=archived',
  created_by BIGINT UNSIGNED DEFAULT NULL,
  created_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  updated_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE
CURRENT_TIMESTAMP(3),
  is_deleted TINYINT NOT NULL DEFAULT 0,
  PRIMARY KEY (id),
  KEY idx_qb_question_type_diff (question_type, difficulty),
  KEY idx_qb_question_chapter (chapter),
  KEY idx_qb_question_status (status),
  CONSTRAINT fk_question_analysis_llm FOREIGN KEY (analysis_llm_call_id)
REFERENCES qb_llm_call(id),
  CONSTRAINT fk_question_creator FOREIGN KEY (created_by) REFERENCES
sys_user(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE qb_question_option (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
```

```
  question_id BIGINT UNSIGNED NOT NULL,
  option_label VARCHAR(8) NOT NULL COMMENT 'A/B/C/D...',
  option_content TEXT NOT NULL,
  is_correct TINYINT NOT NULL DEFAULT 0,
  sort_order INT NOT NULL DEFAULT 0,
  PRIMARY KEY (id),
  KEY idx_qb_option_question (question_id),
  CONSTRAINT fk_option_question FOREIGN KEY (question_id) REFERENCES
qb_question(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE qb_question_tag (
  question_id BIGINT UNSIGNED NOT NULL,
  tag_id BIGINT UNSIGNED NOT NULL,
  created_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  PRIMARY KEY (question_id, tag_id),
  KEY idx_qb_question_tag_tag (tag_id, question_id),
  CONSTRAINT fk_qt_question FOREIGN KEY (question_id) REFERENCES
qb_question(id),
  CONSTRAINT fk_qt_tag FOREIGN KEY (tag_id) REFERENCES qb_tag(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE qb_question_case (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  question_id BIGINT UNSIGNED NOT NULL,
  case_no INT NOT NULL DEFAULT 1,
  input_data LONGTEXT NOT NULL,
  expected_output LONGTEXT NOT NULL,
  case_score INT NOT NULL DEFAULT 0,
  is_sample TINYINT NOT NULL DEFAULT 0,
  PRIMARY KEY (id),
  UNIQUE KEY uk_qb_case_q_no (question_id, case_no),
  CONSTRAINT fk_case_question FOREIGN KEY (question_id) REFERENCES
qb_question(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- --------------------------
-- 2) Paper / Assignment
-- --------------------------
CREATE TABLE qb_paper (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  paper_title VARCHAR(255) NOT NULL,
  paper_desc TEXT DEFAULT NULL,
  paper_type TINYINT NOT NULL DEFAULT 1 COMMENT
```

```
'1=manual,2=rule_generated',
  total_score INT NOT NULL DEFAULT 0,
  rule_json JSON DEFAULT NULL,
  status TINYINT NOT NULL DEFAULT 1 COMMENT
'1=draft,2=published,3=archived',
  creator_id BIGINT UNSIGNED NOT NULL,
  created_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  updated_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE
CURRENT_TIMESTAMP(3),
  is_deleted TINYINT NOT NULL DEFAULT 0,
  PRIMARY KEY (id),
  KEY idx_qb_paper_creator (creator_id),
  CONSTRAINT fk_paper_creator FOREIGN KEY (creator_id) REFERENCES
sys_user(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE qb_paper_question (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  paper_id BIGINT UNSIGNED NOT NULL,
  question_id BIGINT UNSIGNED NOT NULL,
  order_no INT NOT NULL DEFAULT 1,
  score INT NOT NULL DEFAULT 0,
  snapshot_json JSON DEFAULT NULL COMMENT 'question snapshot for
traceability',
  snapshot_hash CHAR(64) DEFAULT NULL,
  PRIMARY KEY (id),
  UNIQUE KEY uk_pq_paper_order (paper_id, order_no),
  KEY idx_pq_paper (paper_id),
  KEY idx_pq_question (question_id),
  CONSTRAINT fk_pq_paper FOREIGN KEY (paper_id) REFERENCES qb_paper(id),
  CONSTRAINT fk_pq_question FOREIGN KEY (question_id) REFERENCES
qb_question(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE qb_assignment (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  paper_id BIGINT UNSIGNED NOT NULL,
  assignment_title VARCHAR(255) NOT NULL,
  assignment_desc TEXT DEFAULT NULL,
  start_time DATETIME(3) DEFAULT NULL,
  end_time DATETIME(3) NOT NULL,
  time_limit_min INT NOT NULL DEFAULT 0,
  max_attempts INT NOT NULL DEFAULT 1,
  shuffle_questions TINYINT NOT NULL DEFAULT 0,
```

```
  shuffle_options TINYINT NOT NULL DEFAULT 0,
  publish_status TINYINT NOT NULL DEFAULT 1 COMMENT
'1=draft,2=published,3=closed',
  created_by BIGINT UNSIGNED NOT NULL,
  created_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  updated_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE
CURRENT_TIMESTAMP(3),
  is_deleted TINYINT NOT NULL DEFAULT 0,
  PRIMARY KEY (id),
  KEY idx_assignment_paper (paper_id),
  KEY idx_assignment_deadline (end_time),
  KEY idx_assignment_creator (created_by),
  CONSTRAINT fk_assignment_paper FOREIGN KEY (paper_id) REFERENCES
qb_paper(id),
  CONSTRAINT fk_assignment_creator FOREIGN KEY (created_by) REFERENCES
sys_user(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE qb_assignment_target (
  assignment_id BIGINT UNSIGNED NOT NULL,
  user_id BIGINT UNSIGNED NOT NULL,
  created_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  PRIMARY KEY (assignment_id, user_id),
  KEY idx_assignment_target_user (user_id),
  CONSTRAINT fk_at_assignment FOREIGN KEY (assignment_id) REFERENCES
qb_assignment(id),
  CONSTRAINT fk_at_user FOREIGN KEY (user_id) REFERENCES sys_user(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- --------------------------
-- 3) Attempt / Answer / Grading
-- --------------------------
CREATE TABLE qb_attempt (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  assignment_id BIGINT UNSIGNED DEFAULT NULL,
  paper_id BIGINT UNSIGNED DEFAULT NULL,
  user_id BIGINT UNSIGNED NOT NULL,
  attempt_type TINYINT NOT NULL COMMENT '1=assignment,2=practice',
  attempt_no INT NOT NULL DEFAULT 1,
  status TINYINT NOT NULL DEFAULT 1 COMMENT
'1=in_progress,2=submitted,3=grading,4=graded',
  started_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  submitted_at DATETIME(3) DEFAULT NULL,
  duration_sec INT DEFAULT NULL,
```

```
  total_score INT NOT NULL DEFAULT 0,
  objective_score INT NOT NULL DEFAULT 0,
  subjective_score INT NOT NULL DEFAULT 0,
  needs_review TINYINT NOT NULL DEFAULT 0,
  created_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  updated_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE
CURRENT_TIMESTAMP(3),
  PRIMARY KEY (id),
  KEY idx_attempt_user_created (user_id, created_at),
  KEY idx_attempt_assignment (assignment_id),
  CONSTRAINT fk_attempt_user FOREIGN KEY (user_id) REFERENCES
sys_user(id),
  CONSTRAINT fk_attempt_assignment FOREIGN KEY (assignment_id) REFERENCES
qb_assignment(id),
  CONSTRAINT fk_attempt_paper FOREIGN KEY (paper_id) REFERENCES
qb_paper(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE qb_attempt_question (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  attempt_id BIGINT UNSIGNED NOT NULL,
  question_id BIGINT UNSIGNED NOT NULL,
  order_no INT NOT NULL DEFAULT 1,
  score INT NOT NULL DEFAULT 0,
  snapshot_json JSON DEFAULT NULL,
  snapshot_hash CHAR(64) DEFAULT NULL,
  question_type TINYINT NOT NULL,
  difficulty TINYINT NOT NULL,
  tag_ids_json JSON DEFAULT NULL,
  PRIMARY KEY (id),
  UNIQUE KEY uk_aq_attempt_order (attempt_id, order_no),
  KEY idx_aq_attempt (attempt_id),
  KEY idx_aq_question (question_id),
  CONSTRAINT fk_aq_attempt FOREIGN KEY (attempt_id) REFERENCES
qb_attempt(id),
  CONSTRAINT fk_aq_question FOREIGN KEY (question_id) REFERENCES
qb_question(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE qb_answer (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  attempt_id BIGINT UNSIGNED NOT NULL,
  attempt_question_id BIGINT UNSIGNED NOT NULL,
  question_id BIGINT UNSIGNED NOT NULL,
```

```
  user_id BIGINT UNSIGNED NOT NULL,
  answer_content LONGTEXT DEFAULT NULL,
  answer_format TINYINT NOT NULL DEFAULT 1,
  answer_status TINYINT NOT NULL DEFAULT 1 COMMENT '1=draft,2=submitted',
  auto_score INT NOT NULL DEFAULT 0,
  final_score INT NOT NULL DEFAULT 0,
  is_correct TINYINT NOT NULL DEFAULT 0,
  answered_at DATETIME(3) DEFAULT NULL,
  graded_at DATETIME(3) DEFAULT NULL,
  PRIMARY KEY (id),
  UNIQUE KEY uk_answer_attempt_question (attempt_question_id),
  KEY idx_answer_attempt_question (attempt_id, question_id),
  CONSTRAINT fk_answer_attempt FOREIGN KEY (attempt_id) REFERENCES
qb_attempt(id),
  CONSTRAINT fk_answer_attempt_question FOREIGN KEY (attempt_question_id)
REFERENCES qb_attempt_question(id),
  CONSTRAINT fk_answer_question FOREIGN KEY (question_id) REFERENCES
qb_question(id),
  CONSTRAINT fk_answer_user FOREIGN KEY (user_id) REFERENCES sys_user(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE qb_grading_record (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  answer_id BIGINT UNSIGNED NOT NULL,
  grading_mode TINYINT NOT NULL COMMENT '1=auto,2=llm,3=manual',
  score INT NOT NULL DEFAULT 0,
  detail_json JSON DEFAULT NULL,
  llm_call_id BIGINT UNSIGNED DEFAULT NULL,
  confidence DECIMAL(6,4) DEFAULT NULL,
  needs_review TINYINT NOT NULL DEFAULT 0,
  reviewer_id BIGINT UNSIGNED DEFAULT NULL,
  review_comment TEXT DEFAULT NULL,
  is_final TINYINT NOT NULL DEFAULT 0,
  created_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  PRIMARY KEY (id),
  KEY idx_gr_answer (answer_id),
  KEY idx_gr_llm (llm_call_id),
  KEY idx_gr_created (created_at),
  CONSTRAINT fk_gr_answer FOREIGN KEY (answer_id) REFERENCES
qb_answer(id),
  CONSTRAINT fk_gr_llm FOREIGN KEY (llm_call_id) REFERENCES
qb_llm_call(id),
  CONSTRAINT fk_gr_reviewer FOREIGN KEY (reviewer_id) REFERENCES
sys_user(id)
```

```
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE qb_appeal (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  answer_id BIGINT UNSIGNED NOT NULL,
  user_id BIGINT UNSIGNED NOT NULL,
  reason_text TEXT NOT NULL,
  appeal_status TINYINT NOT NULL DEFAULT 1 COMMENT
'1=pending,2=approved,3=rejected,4=resolved',
  created_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  handled_by BIGINT UNSIGNED DEFAULT NULL,
  handled_at DATETIME(3) DEFAULT NULL,
  decision_comment TEXT DEFAULT NULL,
  final_score INT DEFAULT NULL,
  PRIMARY KEY (id),
  KEY idx_appeal_answer (answer_id),
  KEY idx_appeal_user (user_id),
  CONSTRAINT fk_appeal_answer FOREIGN KEY (answer_id) REFERENCES
qb_answer(id),
  CONSTRAINT fk_appeal_user FOREIGN KEY (user_id) REFERENCES sys_user(id),
  CONSTRAINT fk_appeal_handler FOREIGN KEY (handled_by) REFERENCES
sys_user(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- --------------------------
-- 4) Statistics / Adaptive
-- --------------------------
CREATE TABLE qb_tag_mastery (
  user_id BIGINT UNSIGNED NOT NULL,
  tag_id BIGINT UNSIGNED NOT NULL,
  mastery_value DECIMAL(6,4) NOT NULL DEFAULT 0.0000,
  correct_count INT NOT NULL DEFAULT 0,
  attempt_count INT NOT NULL DEFAULT 0,
  updated_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE
CURRENT_TIMESTAMP(3),
  PRIMARY KEY (user_id, tag_id),
  KEY idx_mastery_tag (tag_id),
  CONSTRAINT fk_mastery_user FOREIGN KEY (user_id) REFERENCES
sys_user(id),
  CONSTRAINT fk_mastery_tag FOREIGN KEY (tag_id) REFERENCES qb_tag(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE qb_user_ability (
  user_id BIGINT UNSIGNED NOT NULL,
```

```
  ability_score INT NOT NULL DEFAULT 0,
  updated_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE
CURRENT_TIMESTAMP(3),
  PRIMARY KEY (user_id),
  CONSTRAINT fk_ability_user FOREIGN KEY (user_id) REFERENCES sys_user(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE qb_question_user_stat (
  user_id BIGINT UNSIGNED NOT NULL,
  question_id BIGINT UNSIGNED NOT NULL,
  attempt_count INT NOT NULL DEFAULT 0,
  correct_count INT NOT NULL DEFAULT 0,
  last_attempt_at DATETIME(3) DEFAULT NULL,
  PRIMARY KEY (user_id, question_id),
  KEY idx_qus_question (question_id),
  CONSTRAINT fk_qus_user FOREIGN KEY (user_id) REFERENCES sys_user(id),
  CONSTRAINT fk_qus_question FOREIGN KEY (question_id) REFERENCES
qb_question(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE qb_wrong_question (
  user_id BIGINT UNSIGNED NOT NULL,
  question_id BIGINT UNSIGNED NOT NULL,
  wrong_count INT NOT NULL DEFAULT 0,
  first_wrong_at DATETIME(3) DEFAULT NULL,
  last_wrong_at DATETIME(3) DEFAULT NULL,
  is_resolved TINYINT NOT NULL DEFAULT 0,
  resolved_at DATETIME(3) DEFAULT NULL,
  PRIMARY KEY (user_id, question_id),
  KEY idx_wq_question (question_id),
  CONSTRAINT fk_wq_user FOREIGN KEY (user_id) REFERENCES sys_user(id),
  CONSTRAINT fk_wq_question FOREIGN KEY (question_id) REFERENCES
qb_question(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE qb_practice_rule (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  rule_name VARCHAR(128) NOT NULL,
  rule_json JSON NOT NULL,
  is_default TINYINT NOT NULL DEFAULT 0,
  created_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  PRIMARY KEY (id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

```
CREATE TABLE qb_practice_generation (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  user_id BIGINT UNSIGNED NOT NULL,
  attempt_id BIGINT UNSIGNED NOT NULL,
  rule_id BIGINT UNSIGNED DEFAULT NULL,
  scope_json JSON NOT NULL,
  target_diff_min TINYINT NOT NULL,
  target_diff_max TINYINT NOT NULL,
  debug_json JSON DEFAULT NULL,
  created_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  PRIMARY KEY (id),
  KEY idx_pg_user_created (user_id, created_at),
  CONSTRAINT fk_pg_user FOREIGN KEY (user_id) REFERENCES sys_user(id),
  CONSTRAINT fk_pg_attempt FOREIGN KEY (attempt_id) REFERENCES
qb_attempt(id),
  CONSTRAINT fk_pg_rule FOREIGN KEY (rule_id) REFERENCES
qb_practice_rule(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- --------------------------
-- 5) Forum
-- --------------------------
CREATE TABLE qb_post (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  author_id BIGINT UNSIGNED NOT NULL,
  title VARCHAR(255) NOT NULL,
  content LONGTEXT NOT NULL,
  related_question_id BIGINT UNSIGNED DEFAULT NULL,
  post_status TINYINT NOT NULL DEFAULT 1 COMMENT '1=normal,2=closed',
  created_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  updated_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE
CURRENT_TIMESTAMP(3),
  is_deleted TINYINT NOT NULL DEFAULT 0,
  PRIMARY KEY (id),
  KEY idx_post_author_created (author_id, created_at),
  KEY idx_post_related_question (related_question_id),
  CONSTRAINT fk_post_author FOREIGN KEY (author_id) REFERENCES
sys_user(id),
  CONSTRAINT fk_post_related_question FOREIGN KEY (related_question_id)
REFERENCES qb_question(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE qb_reply (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
```

```
  post_id BIGINT UNSIGNED NOT NULL,
  author_id BIGINT UNSIGNED NOT NULL,
  parent_reply_id BIGINT UNSIGNED DEFAULT NULL,
  content LONGTEXT NOT NULL,
  created_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  is_deleted TINYINT NOT NULL DEFAULT 0,
  PRIMARY KEY (id),
  KEY idx_reply_post_created (post_id, created_at),
  KEY idx_reply_parent (parent_reply_id),
  CONSTRAINT fk_reply_post FOREIGN KEY (post_id) REFERENCES qb_post(id),
  CONSTRAINT fk_reply_author FOREIGN KEY (author_id) REFERENCES
sys_user(id),
  CONSTRAINT fk_reply_parent FOREIGN KEY (parent_reply_id) REFERENCES
qb_reply(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE qb_post_tag (
  post_id BIGINT UNSIGNED NOT NULL,
  tag_id BIGINT UNSIGNED NOT NULL,
  created_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  PRIMARY KEY (post_id, tag_id),
  KEY idx_post_tag_tag (tag_id, post_id),
  CONSTRAINT fk_pt_post FOREIGN KEY (post_id) REFERENCES qb_post(id),
  CONSTRAINT fk_pt_tag FOREIGN KEY (tag_id) REFERENCES qb_tag(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- --------------------------
-- 6) Logs
-- --------------------------
CREATE TABLE sys_audit_log (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  user_id BIGINT UNSIGNED DEFAULT NULL,
  action VARCHAR(64) NOT NULL,
  entity_type VARCHAR(64) NOT NULL,
  entity_id BIGINT UNSIGNED DEFAULT NULL,
  before_json JSON DEFAULT NULL,
  after_json JSON DEFAULT NULL,
  ip_addr VARCHAR(64) DEFAULT NULL,
  created_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  PRIMARY KEY (id),
  KEY idx_audit_user_created (user_id, created_at),
  KEY idx_audit_entity (entity_type, entity_id),
  CONSTRAINT fk_audit_user FOREIGN KEY (user_id) REFERENCES sys_user(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

```
CREATE TABLE sys_login_log (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  user_id BIGINT UNSIGNED DEFAULT NULL,
  username VARCHAR(64) DEFAULT NULL,
  success_flag TINYINT NOT NULL DEFAULT 1,
  fail_reason VARCHAR(255) DEFAULT NULL,
  ip_addr VARCHAR(64) DEFAULT NULL,
  user_agent VARCHAR(255) DEFAULT NULL,
  login_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  PRIMARY KEY (id),
  KEY idx_login_user_time (user_id, login_at),
  KEY idx_login_username_time (username, login_at)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

类图
@startuml
skinparam classAttributeIconSize 0

package "domain" {

  class User {
    +id: Long
    +username: String
    +passwordHash: String
    +displayName: String
    +email: String
    +status: int
  }

  class Role {
    +id: Long
    +roleCode: String
    +roleName: String
  }

  class Permission {
    +id: Long
    +permCode: String
    +permName: String
    +resource: String
  }

  class Tag {
```

```
    +id: Long
    +tagName: String
    +tagCode: String
    +parentId: Long
    +tagLevel: int
    +tagType: int
  }

  class Question {
    +id: Long
    +title: String
    +questionType: int
    +difficulty: int
    +chapter: String
    +stem: String
    +standardAnswer: String
    +analysisText: String
    +status: int
  }

  class QuestionOption {
    +id: Long
    +questionId: Long
    +optionLabel: String
    +optionContent: String
    +isCorrect: boolean
  }

  class Paper {
    +id: Long
    +paperTitle: String
    +paperType: int
    +totalScore: int
    +creatorId: Long
  }

  class PaperQuestion {
    +id: Long
    +paperId: Long
    +questionId: Long
    +orderNo: int
    +score: int
    +snapshotJson: String
  }
```

```
class Assignment {
  +id: Long
  +paperId: Long
  +assignmentTitle: String
  +startTime: DateTime
  +endTime: DateTime
  +timeLimitMin: int
  +maxAttempts: int
  +publishStatus: int
}

class Attempt {
  +id: Long
  +assignmentId: Long
  +paperId: Long
  +userId: Long
  +attemptType: int
  +attemptNo: int
  +status: int
  +startedAt: DateTime
  +submittedAt: DateTime
  +totalScore: int
  +needsReview: boolean
}

class AttemptQuestion {
  +id: Long
  +attemptId: Long
  +questionId: Long
  +orderNo: int
  +score: int
  +snapshotJson: String
  +questionType: int
  +difficulty: int
}

class Answer {
  +id: Long
  +attemptId: Long
  +attemptQuestionId: Long
  +questionId: Long
  +userId: Long
  +answerContent: String
```

```
    +answerStatus: int
    +autoScore: int
    +finalScore: int
    +isCorrect: boolean
  }

  class GradingRecord {
    +id: Long
    +answerId: Long
    +gradingMode: int
    +score: int
    +detailJson: String
    +llmCallId: Long
    +confidence: double
    +needsReview: boolean
    +reviewerId: Long
    +isFinal: boolean
  }

  class LlmCall {
    +id: Long
    +bizType: int
    +bizId: Long
    +modelName: String
    +promptText: String
    +responseText: String
    +responseJson: String
    +callStatus: int
  }

  class Appeal {
    +id: Long
    +answerId: Long
    +userId: Long
    +reasonText: String
    +appealStatus: int
    +handledBy: Long
    +finalScore: int
  }

  class TagMastery {
    +userId: Long
    +tagId: Long
    +masteryValue: double
```

```
    +correctCount: int
    +attemptCount: int
  }

  class UserAbility {
    +userId: Long
    +abilityScore: int
    +updatedAt: DateTime
  }

  class QuestionUserStat {
    +userId: Long
    +questionId: Long
    +attemptCount: int
    +correctCount: int
    +lastAttemptAt: DateTime
  }

  class WrongQuestion {
    +userId: Long
    +questionId: Long
    +wrongCount: int
    +lastWrongAt: DateTime
    +isResolved: boolean
  }

  class Post {
    +id: Long
    +authorId: Long
    +title: String
    +content: String
    +relatedQuestionId: Long
    +postStatus: int
  }

  class Reply {
    +id: Long
    +postId: Long
    +authorId: Long
    +parentReplyId: Long
    +content: String
  }
}
```

```
package "service" {

  interface AuthService {
    +login(username:String, password:String): String
    +parseToken(token:String): User
    +checkPermission(userId:Long, perm:String): boolean
  }

  interface TagService {
    +createTag(tag:Tag): Long
    +updateTag(tag:Tag): void
    +deleteTag(tagId:Long): void
    +listTags(): List<Tag>
  }

  interface QuestionService {
    +createQuestion(q:Question, options:List<QuestionOption>,
tagIds:List<Long>): Long
    +updateQuestion(q:Question, options:List<QuestionOption>,
tagIds:List<Long>): void
    +publishQuestion(questionId:Long): void
    +searchQuestions(criteriaJson:String, page:int, size:int):
Page<Question>
    +generateAnalysisByLlm(questionId:Long): LlmCall
  }

  interface PaperService {
    +createPaper(p:Paper): Long
    +addQuestion(paperId:Long, questionId:Long, score:int, orderNo:int):
void
    +removeQuestion(paperQuestionId:Long): void
    +calculateTotalScore(paperId:Long): int
  }

  interface AssignmentService {
    +publishAssignment(a:Assignment): Long
    +closeAssignment(assignmentId:Long): void
    +checkStartAllowed(assignmentId:Long, userId:Long): boolean
  }

  interface AttemptService {
    +startAssignmentAttempt(assignmentId:Long, userId:Long): Attempt
    +startPracticeAttempt(scopeJson:String, userId:Long): Attempt
    +saveAnswerDraft(answer:Answer): void
```

```
    +submitAttempt(attemptId:Long): void
  }

  interface GradingService {
    +autoGradeAttempt(attemptId:Long): void
    +requestLlmGrade(answerId:Long): LlmCall
    +finalizeManualGrade(answerId:Long, score:int, comment:String): void
    +markNeedReview(answerId:Long, reason:String): void
  }

  interface AdaptiveService {
    +generateAdaptiveAttempt(scopeJson:String, userId:Long): Attempt
    +updateMasteryAndAbility(attemptId:Long): void
    +computeQuestionWeight(userId:Long, questionId:Long): double
  }

  interface AppealService {
    +submitAppeal(answerId:Long, userId:Long, reason:String): Long
    +handleAppeal(appealId:Long, teacherId:Long, finalScore:int,
comment:String): void
  }

  interface ForumService {
    +createPost(p:Post, tagIds:List<Long>): Long
    +reply(postId:Long, r:Reply): Long
    +listPosts(tagId:Long, page:int, size:int): Page<Post>
  }
}

' ------------------------
' relationships
' ------------------------
User "many" -- "many" Role
Role "many" -- "many" Permission

Question "1" -- "many" QuestionOption
Question "many" -- "many" Tag
Paper "1" -- "many" PaperQuestion
PaperQuestion --> Question

Assignment --> Paper
Attempt --> Assignment
Attempt "1" -- "many" AttemptQuestion
AttemptQuestion --> Question
```

```
Attempt "1" -- "many" Answer
Answer --> AttemptQuestion
Answer "1" -- "many" GradingRecord
GradingRecord --> LlmCall
Answer "0..1" -- "many" Appeal

TagMastery --> User
TagMastery --> Tag
UserAbility --> User
QuestionUserStat --> User
QuestionUserStat --> Question
WrongQuestion --> User
WrongQuestion --> Question

Post --> User
Post "1" -- "many" Reply
Reply --> User

' service dependencies (simplified)
AuthService ..> User
QuestionService ..> Question
PaperService ..> Paper
AttemptService ..> Attempt
GradingService ..> GradingRecord
AdaptiveService ..> TagMastery
ForumService ..> Post
AppealService ..> Appeal

@enduml
```

附录 B：用例图图片（Use_Case_Diagram.png）

如果你在同一仓库/文档目录下放置 `Use_Case_Diagram.png`，可直接用 Markdown 引用：

```
![](Use_Case_Diagram.png)
```